```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

digits=load_digits()
X=digits.data
y=to_categorical(digits.target,num_classes=10)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

model=keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(32,activation='relu'),
    layers.Dense(16,activation='relu'),
    layers.Dense(10,activation='softmax')
])

model.compile(optimizer='sgd',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(X_train,y_train,epochs=50,batch_size=8,validation_split=0.1,verbose=1)

loss,accuracy=model.evaluate(X_test,y_test,verbose=0)
print("Test Loss:",loss)
print("Test Accuracy:",accuracy)

predictions=model.predict(X_test,verbose=0)
predicted_classes=np.argmax(predictions,axis=1)
actual_classes=np.argmax(y_test,axis=1)

print("Predicted Classes:",predicted_classes)
print("Actual Classes:",actual_classes)
```

```
Epoch 1/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 4s 15ms/step - accuracy: 0.2684 - loss: 2.1042 - val_accuracy: 0.4028 - val_loss: 1.8866
Epoch 2/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.5507 - loss: 1.5523 - val_accuracy: 0.6181 - val_loss: 1.3668
Epoch 3/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 2s 11ms/step - accuracy: 0.7440 - loss: 1.0587 - val_accuracy: 0.7917 - val_loss: 0.8902
Epoch 4/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8523 - loss: 0.6920 - val_accuracy: 0.8611 - val_loss: 0.6097
Epoch 5/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8933 - loss: 0.4789 - val_accuracy: 0.8681 - val_loss: 0.4742
Epoch 6/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9149 - loss: 0.3591 - val_accuracy: 0.8889 - val_loss: 0.3738
Epoch 7/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9343 - loss: 0.2814 - val_accuracy: 0.8958 - val_loss: 0.3312
Epoch 8/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9497 - loss: 0.2291 - val_accuracy: 0.9167 - val_loss: 0.2876
Epoch 9/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9536 - loss: 0.1938 - val_accuracy: 0.9167 - val_loss: 0.2620
Epoch 10/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9637 - loss: 0.1670 - val_accuracy: 0.9306 - val_loss: 0.2495
Epoch 11/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9691 - loss: 0.1458 - val_accuracy: 0.9306 - val_loss: 0.2413
Epoch 12/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9714 - loss: 0.1301 - val_accuracy: 0.9375 - val_loss: 0.2303
Epoch 13/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9760 - loss: 0.1170 - val_accuracy: 0.9375 - val_loss: 0.2198
Epoch 14/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9799 - loss: 0.1053 - val_accuracy: 0.9444 - val_loss: 0.2065
Epoch 15/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9822 - loss: 0.0963 - val_accuracy: 0.9375 - val_loss: 0.2059
Epoch 16/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9853 - loss: 0.0881 - val_accuracy: 0.9306 - val_loss: 0.2113
Epoch 17/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9876 - loss: 0.0816 - val_accuracy: 0.9375 - val_loss: 0.1990
Epoch 18/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9861 - loss: 0.0756 - val_accuracy: 0.9375 - val_loss: 0.1942
Epoch 19/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9884 - loss: 0.0700 - val_accuracy: 0.9375 - val_loss: 0.1918
Epoch 20/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9884 - loss: 0.0650 - val_accuracy: 0.9375 - val_loss: 0.1875
Epoch 21/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9899 - loss: 0.0607 - val_accuracy: 0.9306 - val_loss: 0.1890
Epoch 22/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9892 - loss: 0.0568 - val_accuracy: 0.9375 - val_loss: 0.1841
Epoch 23/50
```

```
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9915 - loss: 0.0530 - val_accuracy: 0.9306 - val_loss: 0.1839
Epoch 24/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9923 - loss: 0.0498 - val_accuracy: 0.9306 - val_loss: 0.1781
Epoch 25/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9946 - loss: 0.0468 - val_accuracy: 0.9306 - val_loss: 0.1764
Epoch 26/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9938 - loss: 0.0441 - val_accuracy: 0.9306 - val_loss: 0.1763
Epoch 27/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9946 - loss: 0.0417 - val_accuracy: 0.9306 - val_loss: 0.1704
Epoch 28/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9946 - loss: 0.0394 - val_accuracy: 0.9306 - val_loss: 0.1748
Epoch 29/50
```

```
Start coding or generate with AI.
```

```
Start coding or generate with AI.
```

```
# Keras program for Handwritten Digits Classification

import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical


digits = load_digits()

X = digits.data
y = digits.target

# conert to one hot encoding
y = to_categorical(y, num_classes=10)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

#  Standardize the input features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create the neural network model
model = keras.Sequential([
    layers.Input(shape=(64,)),
    layers.Dense(32, activation='relu'),
    layers.Dense(16, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(
    optimizer='sgd',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=8,
    validation_split=0.1,
    verbose=1
)

# Evaluate the model
loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("Test Accuracy:", accuracy)
```

```
# Make predictions
predictions = model.predict(X_test)

predicted_classes = np.argmax(predictions, axis=1)
actual_classes = np.argmax(y_test, axis=1)

print("Predicted Classes:", predicted_classes)
print("Actual Classes:", actual_classes)
```

```
Epoch 21/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9954 - loss: 0.0531 - val_accuracy: 0.9583 - val_loss: 0.1081
Epoch 22/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9946 - loss: 0.0490 - val_accuracy: 0.9653 - val_loss: 0.1043
Epoch 23/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9954 - loss: 0.0456 - val_accuracy: 0.9653 - val_loss: 0.1015
Epoch 24/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9961 - loss: 0.0427 - val_accuracy: 0.9722 - val_loss: 0.0994
Epoch 25/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9954 - loss: 0.0398 - val_accuracy: 0.9722 - val_loss: 0.0963
Epoch 26/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9961 - loss: 0.0372 - val_accuracy: 0.9653 - val_loss: 0.0971
Epoch 27/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9969 - loss: 0.0350 - val_accuracy: 0.9722 - val_loss: 0.0928
Epoch 28/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9977 - loss: 0.0327 - val_accuracy: 0.9722 - val_loss: 0.0949
Epoch 29/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9969 - loss: 0.0311 - val_accuracy: 0.9722 - val_loss: 0.0919
Epoch 30/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9977 - loss: 0.0293 - val_accuracy: 0.9722 - val_loss: 0.0916
Epoch 31/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9977 - loss: 0.0275 - val_accuracy: 0.9653 - val_loss: 0.0915
Epoch 32/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9977 - loss: 0.0263 - val_accuracy: 0.9722 - val_loss: 0.0887
Epoch 33/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9977 - loss: 0.0249 - val_accuracy: 0.9722 - val_loss: 0.0872
Epoch 34/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9977 - loss: 0.0236 - val_accuracy: 0.9722 - val_loss: 0.0879
Epoch 35/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9977 - loss: 0.0223 - val_accuracy: 0.9722 - val_loss: 0.0839
Epoch 36/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9985 - loss: 0.0212 - val_accuracy: 0.9722 - val_loss: 0.0836
Epoch 37/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9985 - loss: 0.0203 - val_accuracy: 0.9722 - val_loss: 0.0833
Epoch 38/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9977 - loss: 0.0193 - val_accuracy: 0.9722 - val_loss: 0.0810
Epoch 39/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9985 - loss: 0.0184 - val_accuracy: 0.9722 - val_loss: 0.0814
Epoch 40/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9985 - loss: 0.0177 - val_accuracy: 0.9722 - val_loss: 0.0817
Epoch 41/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9985 - loss: 0.0168 - val_accuracy: 0.9722 - val_loss: 0.0807
Epoch 42/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0161 - val_accuracy: 0.9722 - val_loss: 0.0808
Epoch 43/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9985 - loss: 0.0155 - val_accuracy: 0.9722 - val_loss: 0.0812
Epoch 44/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0148 - val_accuracy: 0.9722 - val_loss: 0.0803
Epoch 45/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0143 - val_accuracy: 0.9722 - val_loss: 0.0804
Epoch 46/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0137 - val_accuracy: 0.9722 - val_loss: 0.0779
Epoch 47/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0132 - val_accuracy: 0.9722 - val_loss: 0.0784
Epoch 48/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0127 - val_accuracy: 0.9722 - val_loss: 0.0785
Epoch 49/50
162/162 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 1.0000 - loss: 0.0122 - val_accuracy: 0.9722 - val_loss: 0.0766
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Flatten, Dense
# ============================================================
# STEP 1: Load MNIST dataset
# ============================================================
(x_train, y_train), (x_test, y_test) =tf.keras.datasets.mnist.load_data()
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Test images:", x_test.shape)
print("Test labels:", y_test.shape)
# ============================================================
# STEP 2: Normalize pixel values
# ============================================================
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# ============================================================
# STEP 3: Create Deep Neural Network
```

```
# ============================================================
model = Sequential([
# INPUT LAYER
Input(shape=(28, 28)),
# Convert 28 x 28 image into 784 values
Flatten(),
# HIDDEN LAYER 1
Dense(128, activation="relu"),
# HIDDEN LAYER 2
Dense(64, activation="relu"),
# OUTPUT LAYER
Dense(10, activation="softmax")
])
# ============================================================
# STEP 4: Display model architecture
# ============================================================
model.summary()
# ============================================================
# STEP 5: Compile the model
# ============================================================
model.compile(
optimizer="adam",
loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)
# ============================================================
# STEP 6: Train the model
# ============================================================
history = model.fit(
x_train,
y_train,
epochs=5,
batch_size=128,
validation_split=0.1
)
# ============================================================
# STEP 7: Evaluate on TEST dataset
# ============================================================
test_loss, test_accuracy = model.evaluate(
x_test,
y_test,
verbose=0
)
print("\n==================================")
print("RESULT WITHOUT AUGMENTATION")
print("==================================")
print("Test Loss     :", test_loss)
print("Test Accuracy :", test_accuracy)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training images: (60000, 28, 28)
Training labels: (60000,)
Test images: (10000, 28, 28)
Test labels: (10000,)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

```
 Total params: 109,386 (427.29 KB)
 Trainable params: 109,386 (427.29 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.8970 - loss: 0.3592 - val_accuracy: 0.9608 - val_loss: 0.1407
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9572 - loss: 0.1437 - val_accuracy: 0.9698 - val_loss: 0.1134
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9695 - loss: 0.1007 - val_accuracy: 0.9765 - val_loss: 0.0932
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9769 - loss: 0.0763 - val_accuracy: 0.9750 - val_loss: 0.0898
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9815 - loss: 0.0616 - val_accuracy: 0.9758 - val_loss: 0.0869

==================================
RESULT WITHOUT AUGMENTATION
==================================
Test Loss     : 0.08078145980834961
```

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import (
Input,
Flatten,
Dense,
RandomRotation,
RandomTranslation,
RandomZoom
)
# ========================================================
# STEP 1: Load MNIST dataset
# ========================================================
(x_train, y_train), (x_test, y_test) =tf.keras.datasets.mnist.load_data()
print("Training images:", x_train.shape)
print("Training labels:", y_train.shape)
print("Test images:", x_test.shape)
print("Test labels:", y_test.shape)
# ========================================================
# STEP 2: Normalize pixel values
# ========================================================
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# ========================================================
# STEP 3: Create DATA AUGMENTATION
# ========================================================
data_augmentation = Sequential([
# Slightly rotate the image
RandomRotation(0.1),
# Slightly move the image
RandomTranslation(
height_factor=0.1,
width_factor=0.1
),
# Slightly zoom the image
RandomZoom(0.1)
])
# ========================================================
# STEP 4: Create Deep Neural Network
# ========================================================
model = Sequential([
# INPUT LAYER
Input(shape=(28, 28)),
# DATA AUGMENTATION
data_augmentation,
# Convert 28 x 28 into 784 values
Flatten(),
# HIDDEN LAYER 1
Dense(128, activation="relu"),
# HIDDEN LAYER 2
Dense(64, activation="relu"),
# OUTPUT LAYER
Dense(10, activation="softmax")
])
# ========================================================
# STEP 5: Display model architecture
# ========================================================
model.summary()
# ========================================================
# STEP 6: Compile the model
# ========================================================
model.compile(
optimizer="adam",
loss="sparse_categorical_crossentropy",
metrics=["accuracy"]
)
# ========================================================
# STEP 7: Train the model
# ========================================================
history = model.fit(
x_train,
y_train,
epochs=5,
batch_size=128,
validation_split=0.1
)
# ========================================================
# STEP 8: Evaluate on TEST dataset
# ========================================================
test_loss, test_accuracy = model.evaluate(
x_test,
y_test,
```

```
verbose=0
)
print("\n==================================")
print("RESULT WITH AUGMENTATION")
print("==================================")
print("Test Loss     :", test_loss)
print("Test Accuracy :", test_accuracy)
```

```
Training images: (60000, 28, 28)
Training labels: (60000,)
Test images: (10000, 28, 28)
Test labels: (10000,)
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
sequential_1 (Sequential)	(None, 28, 28)	0
flatten_1 (Flatten)	(None, 784)	0
dense_3 (Dense)	(None, 128)	100,480
dense_4 (Dense)	(None, 64)	8,256
dense_5 (Dense)	(None, 10)	650

```
 Total params: 109,386 (427.29 KB)
 Trainable params: 109,386 (427.29 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 9s 19ms/step - accuracy: 0.1087 - loss: 2.3039 - val_accuracy: 0.2343 - val_loss: 2.2277
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - accuracy: 0.1131 - loss: 2.3014 - val_accuracy: 0.2240 - val_loss: 2.2438
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 8s 18ms/step - accuracy: 0.1143 - loss: 2.3007 - val_accuracy: 0.2190 - val_loss: 2.2399
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.1132 - loss: 2.3009 - val_accuracy: 0.1947 - val_loss: 2.2430
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 7s 17ms/step - accuracy: 0.1138 - loss: 2.3005 - val_accuracy: 0.2258 - val_loss: 2.2372

==================================
RESULT WITH AUGMENTATION
==================================
Test Loss     : 2.2333824634552
```

```
import tensorflow as tf
import numpy as np

from tensorflow.keras import Sequential
from tensorflow.keras.layers import (
    Input,
    Flatten,
    Dense
)

# ============================================================
# STEP 1: LOAD MNIST DATASET
# ============================================================

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

print("Original training dataset:", x_train.shape)
print("Original test dataset:", x_test.shape)


# ============================================================
# STEP 2: SELECT 10,000 ORIGINAL TRAINING IMAGES
# ============================================================

N = 10000

x_original = x_train[:N]
y_original = y_train[:N]

print("\nSelected original images:", x_original.shape)
print("Selected original labels:", y_original.shape)


# ============================================================
# STEP 3: CREATE ROTATED IMAGES
# ============================================================

# Convert images to floating point
x_float = x_original.astype("float32")

# Add channel dimension
```

```
# Add channel dimension
# From (10000, 28, 28)
# To   (10000, 28, 28, 1)

x_float = x_float[..., np.newaxis]

# Rotate each image
x_rotated = tf.keras.layers.RandomRotation(
    factor=10 / 360,
    fill_mode="constant"
)(x_float, training=True)

# Convert TensorFlow tensor to NumPy array
x_rotated = x_rotated.numpy()

# Remove channel dimension
x_rotated = np.squeeze(x_rotated, axis=-1)

# Convert pixel values back to uint8
x_rotated = np.clip(x_rotated, 0, 255).astype("uint8")

# Labels remain unchanged
y_rotated = y_original.copy()

print("\nRotated images:", x_rotated.shape)
print("Rotated labels:", y_rotated.shape)


# ============================================================
# STEP 4: CREATE SHIFTED IMAGES
# ============================================================

# Shift images horizontally and vertically

x_shifted = tf.keras.layers.RandomTranslation(
    height_factor=0.10,
    width_factor=0.10,
    fill_mode="constant"
)(x_float, training=True)

# Convert TensorFlow tensor to NumPy
x_shifted = x_shifted.numpy()

# Remove channel dimension
x_shifted = np.squeeze(x_shifted, axis=-1)

# Keep pixel values between 0 and 255
x_shifted = np.clip(x_shifted, 0, 255).astype("uint8")

# Labels remain unchanged
y_shifted = y_original.copy()

print("Shifted images:", x_shifted.shape)
print("Shifted labels:", y_shifted.shape)


# ============================================================
# STEP 5: COMBINE ORIGINAL + ROTATED + SHIFTED DATA
# ============================================================

x_augmented = np.concatenate(
    [x_original, x_rotated, x_shifted],
    axis=0
)

y_augmented = np.concatenate(
    [y_original, y_rotated, y_shifted],
    axis=0
)


# ============================================================
# STEP 6: DISPLAY DATASET SIZE
# ============================================================

print("\n========================================")
print("DATASET SIZE AFTER AUGMENTATION")
print("========================================")

print("Original images :", len(x_original))
print("Rotated images  :", len(x_rotated))
print("Shifted images  :", len(x_shifted))
```

```
print("----------------------------------------")

print("Final images    :", len(x_augmented))
print("Final labels    :", len(y_augmented))


# ============================================================
# STEP 7: NORMALIZE THE AUGMENTED DATA
# ============================================================

x_augmented = x_augmented.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0


# ============================================================
# STEP 8: CREATE DEEP NEURAL NETWORK
# ============================================================

model = Sequential([

    # INPUT LAYER
    Input(shape=(28, 28)),

    # Convert 28 x 28 into 784
    Flatten(),

    # HIDDEN LAYER 1
    Dense(
        128,
        activation="relu"
    ),

    # HIDDEN LAYER 2
    Dense(
        64,
        activation="relu"
    ),

    # OUTPUT LAYER
    Dense(
        10,
        activation="softmax"
    )
])


# ============================================================
# STEP 9: DISPLAY ARCHITECTURE
# ============================================================

print("\n========================================")
print("DEEP NEURAL NETWORK")
print("========================================")

model.summary()


# ============================================================
# STEP 10: COMPILE THE MODEL
# ============================================================

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)


# ============================================================
# STEP 11: TRAIN THE MODEL
# ============================================================

history = model.fit(
    x_augmented,
    y_augmented,
    epochs=5,
    batch_size=128,
    validation_split=0.1
)


# ============================================================
```

```
# STEP 12: TEST THE MODEL
# ============================================================

test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)


# ============================================================
# STEP 13: DISPLAY FINAL RESULT
# ============================================================

print("\n========================================")
print("FINAL RESULT")
print("========================================")

print("Test Loss     :", test_loss)
print("Test Accuracy :", test_accuracy)
```

```
Original training dataset: (60000, 28, 28)
Original test dataset: (10000, 28, 28)

Selected original images: (10000, 28, 28)
Selected original labels: (10000,)

Rotated images: (10000, 28, 28)
Rotated labels: (10000,)
Shifted images: (10000, 28, 28)
Shifted labels: (10000,)

========================================
DATASET SIZE AFTER AUGMENTATION
========================================
Original images : 10000
Rotated images  : 10000
Shifted images  : 10000
----------------------------------------
Final images    : 30000
Final labels    : 30000

========================================
DEEP NEURAL NETWORK
========================================
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
flatten_2 (Flatten)	(None, 784)	0
dense_6 (Dense)	(None, 128)	100,480
dense_7 (Dense)	(None, 64)	8,256
dense_8 (Dense)	(None, 10)	650

```
 Total params: 109,386 (427.29 KB)
 Trainable params: 109,386 (427.29 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 5s 13ms/step - accuracy: 0.8161 - loss: 0.6403 - val_accuracy: 0.7903 - val_loss: 0.7168
Epoch 2/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9211 - loss: 0.2720 - val_accuracy: 0.8533 - val_loss: 0.4938
Epoch 3/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9509 - loss: 0.1736 - val_accuracy: 0.8813 - val_loss: 0.4089
Epoch 4/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9629 - loss: 0.1271 - val_accuracy: 0.8960 - val_loss: 0.3569
Epoch 5/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9730 - loss: 0.0945 - val_accuracy: 0.8997 - val_loss: 0.3550

========================================
FINAL RESULT
========================================
Test Loss     : 0.12255492806434631
Test Accuracy : 0.9621000289916992
```

```
import numpy as np
import tensorflow as tf
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler,OneHotEncoder

iris=load_iris()
X=iris.data
y=iris.target.reshape(-1,1)
```

```
encoder=OneHotEncoder(sparse_output=False)
y_encoded=encoder.fit_transform(y)

X_train,X_test,y_train,y_test=train_test_split(X,y_encoded,test_size=0.2,random_state=42)

scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

def create_model():
    model=tf.keras.Sequential([
        tf.keras.layers.Input(shape=(4,)),
        tf.keras.layers.Dense(8,activation='relu'),
        tf.keras.layers.Dense(3,activation='softmax')
    ])
    model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),loss='categorical_crossentropy',metrics=['accuracy
    return model

num_samples=X_train.shape[0]

print("--- Training with Batch Gradient Descent ---")
model_batch=create_model()
model_batch.fit(X_train,y_train,epochs=50,batch_size=num_samples,verbose=0)
loss,acc=model_batch.evaluate(X_test,y_test,verbose=0)
print(f"Batch GD Test Accuracy: {acc:.4f}\n")

print("--- Training with Stochastic Gradient Descent ---")
model_sgd=create_model()
model_sgd.fit(X_train,y_train,epochs=50,batch_size=1,verbose=0)
loss,acc=model_sgd.evaluate(X_test,y_test,verbose=0)
print(f"SGD Test Accuracy: {acc:.4f}\n")

print("--- Training with Mini-Batch Gradient Descent ---")
model_mini=create_model()
model_mini.fit(X_train,y_train,epochs=50,batch_size=16,verbose=0)
loss,acc=model_mini.evaluate(X_test,y_test,verbose=0)
print(f"Mini-Batch GD Test Accuracy: {acc:.4f}\n")
```

```
--- Training with Batch Gradient Descent ---
Batch GD Test Accuracy: 0.7000

--- Training with Stochastic Gradient Descent ---
SGD Test Accuracy: 0.9667

--- Training with Mini-Batch Gradient Descent ---
Mini-Batch GD Test Accuracy: 0.8667
```

```
import numpy as np
from tensorflow import keras
from tensorflow.keras import layers
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.utils import to_categorical

iris=load_iris()
X=iris.data
y=to_categorical(iris.target,num_classes=3)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)

scaler=StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)

model=keras.Sequential([
    layers.Input(shape=(4,)),
    layers.Dense(8,activation='relu'),
    layers.Dense(3,activation='softmax')
])

model.compile(optimizer='sgd',loss='categorical_crossentropy',metrics=['accuracy'])
model.fit(X_train,y_train,epochs=50,batch_size=8,validation_split=0.1,verbose=1)

loss,accuracy=model.evaluate(X_test,y_test,verbose=0)
print("Test Loss:",loss)
print("Test Accuracy:",accuracy)

predictions=model.predict(X_test,verbose=0)
predicted_classes=np.argmax(predictions,axis=1)
actual_classes=np.argmax(y_test,axis=1)
```

```
print("Predicted Classes:",predicted_classes)
print("Actual Classes:",actual_classes)
```

```
Epoch 1/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 2s 48ms/step - accuracy: 0.2407 - loss: 1.6179 - val_accuracy: 0.1667 - val_loss: 1.3104
Epoch 2/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 1s 12ms/step - accuracy: 0.2593 - loss: 1.4047 - val_accuracy: 0.1667 - val_loss: 1.1984
Epoch 3/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.2685 - loss: 1.2582 - val_accuracy: 0.2500 - val_loss: 1.1179
Epoch 4/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.2685 - loss: 1.1517 - val_accuracy: 0.3333 - val_loss: 1.0623
Epoch 5/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.4722 - loss: 1.0701 - val_accuracy: 0.3333 - val_loss: 1.0159
Epoch 6/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.4815 - loss: 1.0013 - val_accuracy: 0.4167 - val_loss: 0.9748
Epoch 7/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 20ms/step - accuracy: 0.5463 - loss: 0.9438 - val_accuracy: 0.5833 - val_loss: 0.9384
Epoch 8/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.6389 - loss: 0.8924 - val_accuracy: 0.6667 - val_loss: 0.9036
Epoch 9/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7778 - loss: 0.8435 - val_accuracy: 0.6667 - val_loss: 0.8705
Epoch 10/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.7870 - loss: 0.8004 - val_accuracy: 0.6667 - val_loss: 0.8411
Epoch 11/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.7870 - loss: 0.7616 - val_accuracy: 0.6667 - val_loss: 0.8130
Epoch 12/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7963 - loss: 0.7252 - val_accuracy: 0.6667 - val_loss: 0.7874
Epoch 13/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7870 - loss: 0.6920 - val_accuracy: 0.6667 - val_loss: 0.7649
Epoch 14/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7963 - loss: 0.6622 - val_accuracy: 0.6667 - val_loss: 0.7448
Epoch 15/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7963 - loss: 0.6355 - val_accuracy: 0.6667 - val_loss: 0.7262
Epoch 16/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7963 - loss: 0.6117 - val_accuracy: 0.6667 - val_loss: 0.7087
Epoch 17/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7963 - loss: 0.5888 - val_accuracy: 0.6667 - val_loss: 0.6928
Epoch 18/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8056 - loss: 0.5686 - val_accuracy: 0.7500 - val_loss: 0.6785
Epoch 19/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8056 - loss: 0.5509 - val_accuracy: 0.7500 - val_loss: 0.6646
Epoch 20/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8056 - loss: 0.5347 - val_accuracy: 0.7500 - val_loss: 0.6516
Epoch 21/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8056 - loss: 0.5197 - val_accuracy: 0.7500 - val_loss: 0.6399
Epoch 22/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.8056 - loss: 0.5060 - val_accuracy: 0.7500 - val_loss: 0.6290
Epoch 23/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8056 - loss: 0.4936 - val_accuracy: 0.7500 - val_loss: 0.6185
Epoch 24/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8056 - loss: 0.4815 - val_accuracy: 0.7500 - val_loss: 0.6085
Epoch 25/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8148 - loss: 0.4710 - val_accuracy: 0.8333 - val_loss: 0.6000
Epoch 26/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8241 - loss: 0.4610 - val_accuracy: 0.8333 - val_loss: 0.5909
Epoch 27/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8241 - loss: 0.4516 - val_accuracy: 0.8333 - val_loss: 0.5826
Epoch 28/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.8241 - loss: 0.4426 - val_accuracy: 0.8333 - val_loss: 0.5747
Epoch 29/50
14/14 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8241 - loss: 0.4345 - val_accuracy: 0.8333 - val_loss: 0.5677
```

```
import tensorflow as tf

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN,Dense,Dropout
from sklearn.preprocessing import MinMaxScaler

uploaded=files.upload()
filename=list(uploaded.keys())[0]
data=pd.read_csv(filename)
data.columns=data.columns.str.strip()

if "Open" in data.columns:
    prices=pd.to_numeric(data["Open"],errors="coerce")
elif "open" in data.columns:
    prices=pd.to_numeric(data["open"],errors="coerce")
else:
    prices=pd.to_numeric(data.iloc[:,1],errors="coerce")

prices=prices.dropna().values.reshape(-1,1)
```

```
if len(prices)<=60:
    raise ValueError("The dataset must contain more than 60 stock-price records.")

split=int(len(prices)*0.8)
scaler=MinMaxScaler(feature_range=(0,1))
scaler.fit(prices[:split])
scaled_data=scaler.transform(prices)

X_train=[]
y_train=[]
for i in range(60,split):
    X_train.append(scaled_data[i-60:i,0])
    y_train.append(scaled_data[i,0])

X_train=np.array(X_train).reshape(-1,60,1)
y_train=np.array(y_train)

model=Sequential([
    tf.keras.Input(shape=(60,1)),
    SimpleRNN(50,return_sequences=True),
    Dropout(0.2),
    SimpleRNN(50,return_sequences=True),
    Dropout(0.2),
    SimpleRNN(50),
    Dropout(0.2),
    Dense(1)
])

model.compile(optimizer="adam",loss="mean_squared_error")
model.fit(X_train,y_train,epochs=20,batch_size=32)

X_test=[]
for i in range(split,len(prices)):
    X_test.append(scaled_data[i-60:i,0])

X_test=np.array(X_test).reshape(-1,60,1)
predicted_stock_price=model.predict(X_test,verbose=0)
predicted_stock_price=scaler.inverse_transform(predicted_stock_price)
real_stock_price=prices[split:]

print("Predicted Stock Prices:")
print(predicted_stock_price)

plt.figure(figsize=(12,6))
plt.plot(real_stock_price,color="red",label="Actual Google Stock Price")
plt.plot(predicted_stock_price,color="blue",label="Predicted Google Stock Price")
plt.title("Google Stock Price Prediction using RNN")
plt.xlabel("Time")
plt.ylabel("Stock Price")
plt.legend()
plt.show()
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving GOOGL.csv to GOOGL (3).csv
Epoch 1/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 9s 49ms/step - loss: 0.1142
Epoch 2/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 11s 55ms/step - loss: 0.0249
Epoch 3/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 4s 41ms/step - loss: 0.0139
Epoch 4/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 46ms/step - loss: 0.0094
Epoch 5/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 51ms/step - loss: 0.0073
Epoch 6/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 42ms/step - loss: 0.0055
Epoch 7/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 56ms/step - loss: 0.0046
Epoch 8/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 42ms/step - loss: 0.0044
Epoch 9/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 4s 41ms/step - loss: 0.0037
Epoch 10/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 56ms/step - loss: 0.0032
Epoch 11/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 41ms/step - loss: 0.0031
Epoch 12/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 7s 55ms/step - loss: 0.0029
Epoch 13/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 42ms/step - loss: 0.0024
Epoch 14/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 45ms/step - loss: 0.0024
Epoch 15/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 55ms/step - loss: 0.0020
Epoch 16/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 9s 46ms/step - loss: 0.0022
Epoch 17/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 51ms/step - loss: 0.0020
Epoch 18/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 42ms/step - loss: 0.0016
Epoch 19/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 6s 56ms/step - loss: 0.0016
Epoch 20/20
109/109 ━━━━━━━━━━━━━━━━━━━━ 5s 41ms/step - loss: 0.0015
Predicted Stock Prices:
[[1146.959  ]
 [1147.8647 ]
 [1146.1724 ]
 [1146.2714 ]
 [1144.1782 ]
 [1149.7196 ]
 [1149.2345 ]
 [1143.4832 ]
 [1145.1587 ]
 [1157.6858 ]
 [1159.8121 ]
 [1172.4475 ]
 [1174.7623 ]
 [1178.345  ]
 [1175.0408 ]
 [1174.917  ]
 [1156.394  ]
 [1145.5442 ]
 [1126.6492 ]
 [1103.4626 ]
 [1089.0697 ]
 [1083.1042 ]
 [1083.6796 ]
 [1083.9282 ]
 [1093.8934 ]
 [1091.8026 ]
 [1088.7908 ]
 [1080.2144 ]
 [1087.7803 ]
 [1070.9244 ]
 [1062.9144 ]
 [1047.2876 ]
 [1035.5881 ]
 [1033.4532 ]
 [1032.7983 ]
 [1054.7247 ]
 [1052.5353 ]
 [1046.848  ]
 [1050.0092 ]
 [1061.0425 ]
 [1065.7704 ]
 [1069.6672 ]
 [1045.0681 ]
 [1049.1635 ]
 [1031.0376 ]
 [1037.3613 ]
```

```
       [1038.8994 ]
       [1023.9277 ]
       [1015.60455]
       [1010.23047]
       [1017.4075 ]
       [1023.92   ]
       [1027.9147 ]
       [1032.6924 ]
       [1054.4583 ]
       [1071.6306 ]
       [1086.8416 ]
       [1073.2026 ]
       [1064.7854 ]
       [1039.4148 ]
       [1046.6063 ]
       [1038.9647 ]
       [1044.7222 ]
       [1037.5345 ]
       [1031.1863 ]
       [1029.158  ]
       [1038.479  ]
       [1025.6879 ]
       [1024.9764 ]
       [ 996.04816]
       [ 985.34955]
       [ 980.8066 ]
       [ 996.1346 ]
       [1010.03204]
       [1013.79675]
       [1017.9753 ]
       [1026.5106 ]
       [1039.5072 ]
       [1050.8644 ]
       [1061.9612 ]
       [1056.2972 ]
       [1054.9103 ]
       [1044.0126 ]
       [1051.2344 ]
       [1048.3958 ]
       [1061.1022 ]
       [1066.2428 ]
       [1071.8206 ]
       [1070.8284 ]
       [1069.368  ]
       [1067.2842 ]
       [1069.8486 ]
       [1069.7003 ]
       [1063.5619 ]
       [1067.1235 ]
       [1072.923  ]
       [1087.9214 ]
       [1089.0771 ]
       [1111.1322 ]
       [1105.806  ]
       [1098.1998 ]
       [1085.4089 ]
       [1087.6243 ]
       [1087.9922 ]
       [1095.0817 ]
       [1097.4576 ]
       [1097.2588 ]
       [1100.7083 ]
       [1099.8488 ]
       [1101.324  ]
       [1099.5212 ]
       [1094.5948 ]
       [1086.3171 ]
       [1086.9594 ]
       [1089.2297 ]
       [1105.7314 ]
       [1113.6606 ]
       [1131.6246 ]
       [1136.3903 ]
       [1132.2966 ]
       [1126.583  ]
       [1134.9133 ]
       [1148.9357 ]
       [1156.05   ]
       [1161.5547 ]
       [1161.6125 ]
       [1164.8384 ]
       [1169.6815 ]
       [1181.5161 ]
       [1187.3286 ]
       [1185.7621 ]
       [1173.2347 ]
       [1167.8699 ]
       [1162.9136 ]
       [1156.5167 ]
       [1155.0339 ]
```

```
[1158.2845 ]
[1162.5068 ]
[1170.5306 ]
[1176.7626 ]
[1178.4459 ]
[1181.2168 ]
[1175.3696 ]
[1177.372  ]
[1179.323  ]
[1180.0289 ]
[1182.6342 ]
[1191.5778 ]
[1199.4406 ]
[1205.8796 ]
[1210.8142 ]
[1219.241  ]
[1229.8024 ]
[1232.3374 ]
[1236.9727 ]
[1217.8159 ]
[1195.72   ]
[1168.2026 ]
[1162.8743 ]
[1144.686  ]
[1146.9146 ]
[1136.1425 ]
[1135.9423 ]
[1131.6302 ]
[1133.4293 ]
[1125.8286 ]
[1120.8965 ]
[1115.633  ]
[1122.6876 ]
[1127.4541 ]
[1120.4028 ]
[1122.6985 ]
[1120.0133 ]
[1125.1514 ]
[1123.044  ]
[1122.6859 ]
[1103.9016 ]
[1098.7573 ]
[1073.2942 ]
[1059.0895 ]
[1042.7555 ]
[1032.5597 ]
[1026.6189 ]
[1031.2299 ]
[1043.7742 ]
[1051.0095 ]
[1054.3291 ]
[1057.6936 ]
[1069.3163 ]
[1071.7832 ]
[1080.2839 ]
[1085.9252 ]
[1085.2275 ]
[1087.2874 ]
[1093.2483 ]
[1088.8344 ]
[1082.2305 ]
[1070.7771 ]
[1069.1371 ]
[1070.944  ]
[1078.432  ]
[1081.9586 ]
[1088.7277 ]
[1092.0725 ]
[1099.8307 ]
[1107.2891 ]
[1117.3413 ]
[1114.1761 ]
[1116.0276 ]
[1115.1572 ]
[1117.8307 ]
[1120.0511 ]
[1119.641  ]
[1119.0967 ]
[1116.9484 ]
[1115.0894 ]
[1134.0631 ]
[1165.2654 ]
[1182.6991 ]
[1187.7946 ]
[1191.4335 ]
[1191.2893 ]
[1176.8846 ]
[1164.6838 ]
[1145.6754 ]
[1138.9326 ]
[1141.4376 ]
```

```
[1144.3092 ]
[1148.7754 ]
[1150.4121 ]
[1148.434  ]
[1153.7452 ]
[1158.5664 ]
[1162.2089 ]
[1156.0872 ]
[1153.3921 ]
[1154.4005 ]
[1149.4136 ]
[1148.4366 ]
[1145.4717 ]
[1148.1477 ]
[1149.9043 ]
[1157.1699 ]
[1153.2782 ]
[1156.1646 ]
[1161.7119 ]
[1172.335  ]
[1173.8894 ]
[1178.5228 ]
[1175.5292 ]
[1185.0851 ]
[1190.6348 ]
[1193.0735 ]
[1196.307  ]
[1195.2698 ]
[1195.0259 ]
[1199.8114 ]
[1204.4938 ]
[1197.5137 ]
[1199.845  ]
[1198.9717 ]
[1203.1455 ]
[1194.7981 ]
[1185.7421 ]
[1170.3337 ]
[1162.3115 ]
[1163.7228 ]
[1164.2052 ]
[1160.4608 ]
[1162.6714 ]
[1167.7413 ]
[1176.4963 ]
[1189.6749 ]
[1189.7717 ]
[1201.8325 ]
[1209.131  ]
[1213.594  ]
[1213.1472 ]
[1213.9939 ]
[1211.0387 ]
[1217.0319 ]
[1223.466  ]
[1230.1183 ]
[1230.1133 ]
[1224.8239 ]
[1227.4442 ]
[1233.2494 ]
[1241.0453 ]
[1243.9045 ]
[1247.832  ]
[1249.44   ]
[1258.3594 ]
[1258.6323 ]
[1261.0598 ]
[1258.5115 ]
[1257.9625 ]
[1267.54   ]
[1277.4208 ]
[1274.9697 ]
[1270.2621 ]
[1266.2865 ]
[1262.2646 ]
[1266.4099 ]
[1266.066  ]
[1264.7062 ]
[1259.4144 ]
[1251.3986 ]
[1253.3359 ]
[1263.0421 ]
[1274.2532 ]
[1280.9116 ]
[1285.8188 ]
[1293.5303 ]
[1296.8385 ]
[1300.9498 ]
[1305.3616 ]
[1308.3691 ]
```